

UAS MKD

1. Rini Rashifah
2. Evreki Wan Refanja
3. Dian Ardiansah
4. Juan Novriandi Ramadhan

Double-click (or enter) to edit

```
# =====
# 🍌 1. SETUP DAN IMPORT LIBRARIES
# =====

# Install library yang diperlukan
!pip install kagglehub tabulate seaborn plotly scikit-learn

# Import library utama
import kagglehub
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import warnings
from tabulate import tabulate
import os
from datetime import datetime

# Import library untuk machine learning
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.decomposition import PCA

# Import library untuk statistical testing
from scipy import stats
from scipy.stats import chi2_contingency, normaltest, jarque_bera

# Setup untuk visualisasi
plt.style.use('default')
sns.set_palette("husl")
warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)

print("✅ Semua library berhasil diimport!")

Requirement already satisfied: kagglehub in /usr/local/lib/python3.11/dist-packages (0.3.12)
Requirement already satisfied: tabulate in /usr/local/lib/python3.11/dist-packages (0.9.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.11/dist-packages (0.13.2)
Requirement already satisfied: plotly in /usr/local/lib/python3.11/dist-packages (5.24.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-packages (1.6.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from kagglehub) (24.2)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.11/dist-packages (from kagglehub) (6.0.2)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from kagglehub) (2.32.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from kagglehub) (4.67.1)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.11/dist-packages (from seaborn) (2.0.2)
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.11/dist-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.11/dist-packages (from seaborn) (3.10.0)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from plotly) (9.1.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.15.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.2.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->kagglehub) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->kagglehub) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->kagglehub) (2.4.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->kagglehub) (2025.6.15)
```

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.6.0) (1.16.0)
 ✓ Semua library berhasil diimport!

```
# =====
# 📁 2. DATA LOADING DAN INITIAL SETUP
# =====

print("🎵 MENGUNDUH DATASET SPOTIFY GLOBAL STREAMING DATA 2024...")
print("="*80)

# Download dataset dari Kaggle
try:
    path = kagglehub.dataset_download("atharvasoundankar/spotify-global-streaming-data-2024")
    print(f"✓ Dataset berhasil didownload!")
    print(f"📁 Path: {path}")

    # Lihat file yang tersedia
    files = os.listdir(path)
    print(f"📁 File yang tersedia: {files}")

    # Load dataset
    csv_files = [f for f in files if f.endswith('.csv')]
    if csv_files:
        main_file = csv_files[0]
        df_original = pd.read_csv(os.path.join(path, main_file))
        print(f"✓ Dataset berhasil dimuat: {main_file}")
        print(f"📐 Shape: {df_original.shape}")
    else:
        raise Exception("Tidak ada file CSV ditemukan!")

except Exception as e:
    print(f"✗ Error: {e}")
    print(f"📄 Silakan upload file CSV secara manual dan ganti dengan nama file yang sesuai:")
    # df_original = pd.read_csv('nama_file.csv')

# Buat copy untuk preprocessing
df = df_original.copy()

print(f"\n📄 Dataset Summary:")
print(f"  - Jumlah baris: {df.shape[0]:,}")
print(f"  - Jumlah kolom: {df.shape[1]:,}")
print(f"  - Memory usage: {df.memory_usage(deep=True).sum() / 1024**2:.2f} MB")

🔄 🎵 MENGUNDUH DATASET SPOTIFY GLOBAL STREAMING DATA 2024...
=====
Downloading from https://www.kaggle.com/api/v1/datasets/download/atharvasoundankar/spotify-global-streaming-data-2024?dataset\_version=2
100%|██████████| 27.4k/27.4k [00:00<00:00, 24.0MB/s]Extracting files...
✓ Dataset berhasil didownload!
📁 Path: /root/.cache/kagglehub/datasets/atharvasoundankar/spotify-global-streaming-data-2024/versions/2
📁 File yang tersedia: ['Cleaned_Spotify_2024_Global_Streaming_Data.csv', 'Spotify_2024_Global_Streaming_Data.csv']
✓ Dataset berhasil dimuat: Cleaned_Spotify_2024_Global_Streaming_Data.csv
📐 Shape: (500, 12)

📄 Dataset Summary:
  - Jumlah baris: 500
  - Jumlah kolom: 12
  - Memory usage: 0.18 MB

# =====
# 🔍 3. PEMAHAMAN AWAL (DATA UNDERSTANDING)
# =====

print("\n" + "="*80)
print("🔍 PEMAHAMAN AWAL DATASET")
print("="*80)

# 3.1 Informasi Dasar Dataset
print("\n📄 3.1 INFORMASI DASAR DATASET")
print("="*50)

# Dimensi dataset
dimension_data = [
    ["Jumlah Baris", f"{df.shape[0]:,}"],
    ["Jumlah Kolom", f"{df.shape[1]:,}"],
    ["Total Sel", f"{df.shape[0] * df.shape[1]:,}"],
    ["Memory Usage", f"{df.memory_usage(deep=True).sum() / 1024**2:.2f} MB"]
]
print(tabulate(dimension_data, headers=["Metric", "Value"], tablefmt="grid"))
```



PEMAHAMAN AWAL DATASET

3.1 INFORMASI DASAR DATASET

Metric	Value
Jumlah Baris	500
Jumlah Kolom	12
Total Sel	6,000
Memory Usage	0.18 MB

3.2 Info Kolom dan Tipe Data

```
print("\n 3.2 INFORMASI KOLOM DAN TIPE DATA")
print("="*50)
```

```
# Tampilkan info dataset
```

```
print("Dataset Info:")
```

```
df.info()
```

```
# Tabel detail kolom
```

```
column_info = []
```

```
for i, (col, dtype) in enumerate(zip(df.columns, df.dtypes), 1):
```

```
    non_null = df[col].count()
```

```
    null_count = df[col].isnull().sum()
```

```
    null_pct = (null_count / len(df)) * 100
```

```
    column_info.append([
```

```
        i, col, str(dtype), f"{non_null:,.}", f"{null_count:,.}", f"{null_pct:.1f}%"
```

```
    ])
```

```
print(f"\nTabel Detail Kolom:")
```

```
print(tabulate(column_info,
```

```
    headers=["No", "Nama Kolom", "Tipe Data", "Non-Null", "Null", "% Null"],
```

```
    tablefmt="grid"))
```



3.2 INFORMASI KOLOM DAN TIPE DATA

```
Dataset Info:
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 500 entries, 0 to 499
```

```
Data columns (total 12 columns):
```

#	Column	Non-Null Count	Dtype
0	Country	500 non-null	object
1	Artist	500 non-null	object
2	Album	500 non-null	object
3	Genre	500 non-null	object
4	Release Year	500 non-null	int64
5	Monthly Listeners (Millions)	500 non-null	float64
6	Total Streams (Millions)	500 non-null	float64
7	Total Hours Streamed (Millions)	500 non-null	float64
8	Avg Stream Duration (Min)	500 non-null	float64
9	Platform Type	500 non-null	object
10	Streams Last 30 Days (Millions)	500 non-null	float64
11	Skip Rate (%)	500 non-null	float64

```
dtypes: float64(6), int64(1), object(5)
```

```
memory usage: 47.0+ KB
```

```
Tabel Detail Kolom:
```

No	Nama Kolom	Tipe Data	Non-Null	Null	% Null
1	Country	object	500	0	0.0%
2	Artist	object	500	0	0.0%
3	Album	object	500	0	0.0%
4	Genre	object	500	0	0.0%
5	Release Year	int64	500	0	0.0%
6	Monthly Listeners (Millions)	float64	500	0	0.0%
7	Total Streams (Millions)	float64	500	0	0.0%
8	Total Hours Streamed (Millions)	float64	500	0	0.0%

9	Avg Stream Duration (Min)	float64	500	0	0.0%
10	Platform Type	object	500	0	0.0%
11	Streams Last 30 Days (Millions)	float64	500	0	0.0%
12	Skip Rate (%)	float64	500	0	0.0%

```
# 3.3 Preview Data
```

```
print("\n 3.3 PREVIEW DATA")
```

```
print("="*50)
```

```
print("5 Baris Pertama:")
```

```
print(tabulate(df.head(), headers=df.columns, tablefmt="grid", showindex=True))
```

```
print(f"\n5 Baris Terakhir:")
```

```
print(tabulate(df.tail(), headers=df.columns, tablefmt="grid", showindex=True))
```



```
3.3 PREVIEW DATA
```

```
=====
```

```
5 Baris Pertama:
```

	Country	Artist	Album	Genre	Release Year	Monthly Listeners (Millions)	Total Streams (Millions)
0	Germany	Taylor Swift	1989 (Taylor's Version)	K-pop	2020	23.1	
1	Brazil	The Weeknd	After Hours	R&B	2023	60.6	
2	United States	Post Malone	Austin	Reggaeton	2019	42.84	
3	Italy	Ed Sheeran	Autumn Variations	K-pop	2019	73.24	
4	Italy	Ed Sheeran	Autumn Variations	R&B	2019	7.89	

```
5 Baris Terakhir:
```

	Country	Artist	Album	Genre	Release Year	Monthly Listeners (Millions)	Total Streams (Millions)
495	Brazil	Karol G	MAÑANA SERÁ BONITO	Jazz	2019	18.8	
496	Canada	Dua Lipa	Future Nostalgia	Classical	2023	89.68	
497	Germany	Karol G	MAÑANA SERÁ BONITO	Rock	2019	36.93	
498	Canada	SZA	SOS	Indie	2022	87.26	
499	Sweden	BTS	Proof	Reggaeton	2023	89.96	

```
# 3.4 Statistik Deskriptif
```

```
print("\n 3.4 STATISTIK DESKRIPTIF")
```

```
print("="*50)
```

```
# Untuk kolom numerik
```

```
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
```

```
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
```

```
print(f"Kolom Numerik: {len(numeric_cols)} kolom")
```

```
print(f"Kolom Kategorikal: {len(categorical_cols)} kolom")
```

```
if numeric_cols:
```

```
    print(f"\nStatistik Deskriptif Kolom Numerik:")
```

```
    numeric_stats = df[numeric_cols].describe()
```

```
    print(tabulate(numeric_stats.round(2), headers=numeric_stats.columns, tablefmt="grid"))
```



```
3.4 STATISTIK DESKRIPTIF
```

```
=====
```

```
Kolom Numerik: 7 kolom
```

```
Kolom Kategorikal: 5 kolom
```

```
Statistik Deskriptif Kolom Numerik:
```

	Release Year	Monthly Listeners (Millions)	Total Streams (Millions)	Total Hours Streamed (Millions)	Avg S
count	500	500	500	500	
mean	2020.63	51.04	2581.15	8954.37	
std	1.72	28.24	1416.06	5167.65	

min	2018		1.01		53.56		184.3	
+-----+		+-----+		+-----+		+-----+		+-----+
25%	2019		27.67		1337.34		4322.1	
+-----+		+-----+		+-----+		+-----+		+-----+
50%	2020		50.82		2697.35		9053.67	
+-----+		+-----+		+-----+		+-----+		+-----+
75%	2022		75.1		3798.2		12690.2	
+-----+		+-----+		+-----+		+-----+		+-----+
max	2023		99.8		4985.54		21874.9	
+-----+		+-----+		+-----+		+-----+		+-----+

3.5 Klasifikasi dan Summary Kolom

```
print("\n👉 3.5 SUMMARY KOLOM KATEGORIKAL")
```

```
print("="*50)
```

```
if categorical_cols:
    cat_summary = []
    for col in categorical_cols:
        unique_count = df[col].nunique()
        most_common = df[col].mode().iloc[0] if not df[col].mode().empty else "N/A"
        most_common_count = df[col].value_counts().iloc[0] if len(df[col].value_counts()) > 0 else 0

        cat_summary.append([
            col,
            f"{unique_count:,}",
            str(most_common)[:30] + "..." if len(str(most_common)) > 30 else str(most_common),
            f"{most_common_count:,}",
            f"{(most_common_count/len(df)*100):.1f}%"
        ])

print(tabulate(cat_summary,
               headers=["Kolom", "Unique Values", "Nilai Terbanyak", "Frekuensi", "%"],
               tablefmt="grid"))
```



👉 3.5 SUMMARY KOLOM KATEGORIKAL

```
=====
```

Kolom	Unique Values	Nilai Terbanyak	Frekuensi	%	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
Country	20	Italy	34	6.8%	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
Artist	15	BTS	44	8.8%	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
Album	15	Proof	44	8.8%	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
Genre	10	Classical	72	14.4%	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
Platform Type	2	Free	250	50.0%	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+

```
# =====
# 🗂 4. PEMBERSIHAN DATA (DATA CLEANING)
# =====
```

```
print("\n" + "="*80)
print("🗂 PEMBERSIHAN DATA")
print("="*80)
```

4.1 Analisis Missing Values

```
print("\n❌ 4.1 ANALISIS MISSING VALUES")
```

```
print("="*50)
```

```
# Hitung missing values
missing_summary = df.isnull().sum()
missing_summary = missing_summary[missing_summary > 0].sort_values(ascending=False)

if len(missing_summary) > 0:
    missing_detail = []
    for col, missing_count in missing_summary.items():
        missing_pct = (missing_count / len(df)) * 100
        status = "🔴 Tinggi" if missing_pct > 50 else "🟡 Sedang" if missing_pct > 10 else "🟢 Rendah"
        missing_detail.append([col, f"{missing_count:,}", f"{missing_pct:.2f}%", status])

print("Detail Missing Values:")
print(tabulate(missing_detail,
               headers=["Kolom", "Jumlah Missing", "Persentase", "Status"],
               tablefmt="grid"))

# Visualisasi missing values
if len(missing_summary) <= 20: # Hanya jika tidak terlalu banyak kolom
```

```

plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
missing_pct = (missing_summary / len(df)) * 100
plt.barh(range(len(missing_pct)), missing_pct.values)
plt.yticks(range(len(missing_pct)), missing_pct.index)
plt.xlabel('Persentase Missing (%)')
plt.title('Missing Values per Kolom')
plt.gca().invert_yaxis()

plt.subplot(1, 2, 2)
sns.heatmap(df[missing_summary.index].isnull(), cbar=True, yticklabels=False)
plt.title('Pola Missing Values')

plt.tight_layout()
plt.show()

else:
    print("✅ Tidak ada missing values dalam dataset!")

```

=====

PEMBERSIHAN DATA

=====

❌ 4.1 ANALISIS MISSING VALUES

=====

✅ Tidak ada missing values dalam dataset!

4.2 Strategi Penanganan Missing Values

```

print("\n 4.2 STRATEGI PENANGANAN MISSING VALUES")
print("="*50)

def handle_missing_values(df):
    """
    Fungsi untuk menangani missing values dengan strategi yang tepat
    """
    df_cleaned = df.copy()

    for col in df.columns:
        missing_count = df[col].isnull().sum()
        missing_pct = (missing_count / len(df)) * 100

        if missing_count > 0:
            if missing_pct > 70:
                # Drop kolom jika missing > 70%
                df_cleaned = df_cleaned.drop(columns=[col])
                print(f"🗑 Dropped kolom '{col}' (missing {missing_pct:.1f}%)")

            elif df[col].dtype in ['object']:
                # Untuk kategorikal: isi dengan mode atau 'Unknown'
                mode_val = df[col].mode().iloc[0] if not df[col].mode().empty else 'Unknown'
                df_cleaned[col].fillna(mode_val, inplace=True)
                print(f"🔄 Filled '{col}' dengan mode: '{mode_val}'")

            elif df[col].dtype in ['int64', 'float64']:
                # Untuk numerik: isi dengan median
                median_val = df[col].median()
                df_cleaned[col].fillna(median_val, inplace=True)
                print(f"🔄 Filled '{col}' dengan median: {median_val}")

    return df_cleaned

# Terapkan strategi penanganan missing values
if len(missing_summary) > 0:
    df_cleaned = handle_missing_values(df)
    print(f"\n✅ Missing values berhasil ditangani!")
    print(f"  Shape sebelum: {df.shape}")
    print(f"  Shape sesudah: {df_cleaned.shape}")
    df = df_cleaned
else:
    print("✅ Tidak perlu penanganan missing values")

```

=====

4.2 STRATEGI PENANGANAN MISSING VALUES

=====

✅ Tidak perlu penanganan missing values

4.3 Deteksi dan Penanganan Outlier

```

print("\n 4.3 DETEKSI DAN PENANGANAN OUTLIER")
print("="*50)

```

```

def detect_outliers_iqr(df, column):
    """Deteksi outlier menggunakan metode IQR"""
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[column] < lower_bound) | (df[column] > upper_bound)]
    return outliers, lower_bound, upper_bound

# Deteksi outlier untuk kolom numerik
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
outlier_summary = []

for col in numeric_cols:
    outliers, lower, upper = detect_outliers_iqr(df, col)
    outlier_count = len(outliers)
    outlier_pct = (outlier_count / len(df)) * 100

    outlier_summary.append([
        col,
        f"{outlier_count:}",
        f"{outlier_pct:.2f}%",
        f"{lower:.2f}",
        f"{upper:.2f}"
    ])

if outlier_summary:
    print("Deteksi Outlier (Metode IQR):")
    print(tabulate(outlier_summary,
        headers=["Kolom", "Jumlah Outlier", "%", "Lower Bound", "Upper Bound"],
        tablefmt="grid"))

# Visualisasi outlier dengan boxplot
if len(numeric_cols) > 0:
    n_cols = min(3, len(numeric_cols))
    n_rows = (len(numeric_cols) + n_cols - 1) // n_cols

    plt.figure(figsize=(15, 5 * n_rows))
    for i, col in enumerate(numeric_cols):
        plt.subplot(n_rows, n_cols, i + 1)
        plt.boxplot(df[col].dropna())
        plt.title(f'Boxplot {col}')
        plt.ylabel(col)

    plt.tight_layout()
    plt.show()

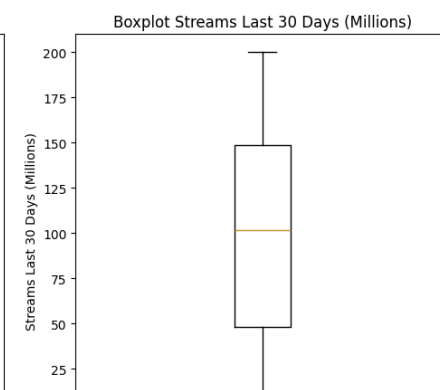
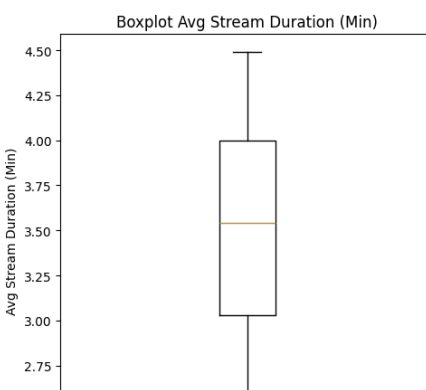
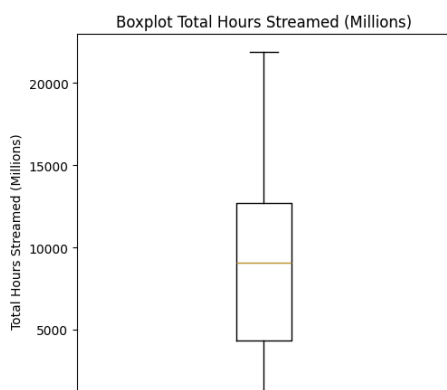
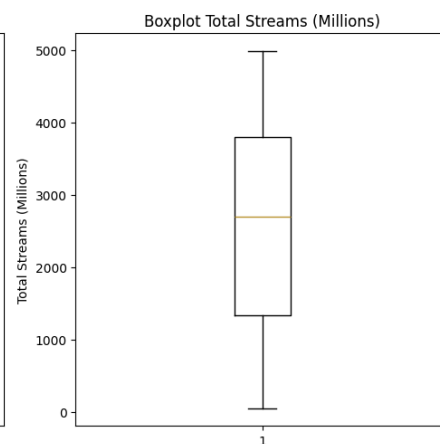
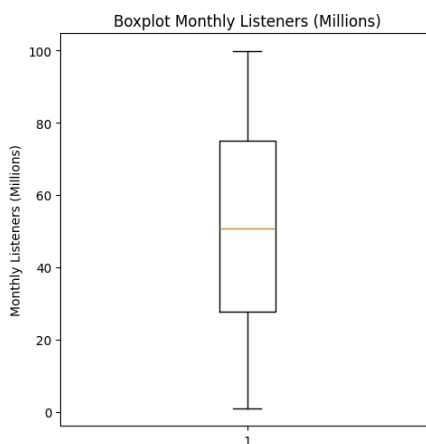
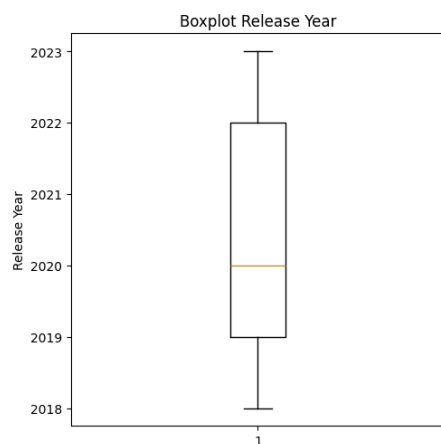
```



4.3 DETEKSI DAN PENANGANAN OUTLIER

Deteksi Outlier (Metode IQR):

Kolom	Jumlah Outlier	%	Lower Bound	Upper Bound
Release Year	0	0.00%	2014.5	2026.5
Monthly Listeners (Millions)	0	0.00%	-43.46	146.23
Total Streams (Millions)	0	0.00%	-2353.95	7489.49
Total Hours Streamed (Millions)	0	0.00%	-8230.03	25242.3
Avg Stream Duration (Min)	0	0.00%	1.57	5.46
Streams Last 30 Days (Millions)	0	0.00%	-102.47	298.94
Skip Rate (%)	0	0.00%	-15.26	55.17



```
# 4.4 Penanganan Data Duplikat
print("\n 4.4 PENANGANAN DATA DUPLIKAT")
print("="*50)

# Deteksi duplikat
duplicate_count = df.duplicated().sum()
duplicate_pct = (duplicate_count / len(df)) * 100

duplicate_stats = [
    ["Total Baris", f"{len(df):,}"],
    ["Baris Duplikat", f"{duplicate_count:,}"],
    ["Persentase Duplikat", f"{duplicate_pct:.2f}%"],
    ["Baris Unik", f"{len(df) - duplicate_count:,}"]
]

print("Statistik Duplikat:")
print(tabulate(duplicate_stats, headers=["Metric", "Value"], tablefmt="grid"))

# Hapus duplikat jika ada
if duplicate_count > 0:
    df_no_dup = df.drop_duplicates()
    print(f"\n  Duplikat dihapus:")
    print(f"   Shape sebelum: {df.shape}")
    print(f"   Shape sesudah: {df_no_dup.shape}")
    df = df_no_dup
else:
    print("\n  Tidak ada data duplikat")
```



4.4 PENANGANAN DATA DUPLIKAT

Statistik Duplikat:

Metric	Value
Total Baris	500
Baris Duplikat	0
Persentase Duplikat	0.00%
Baris Unik	500

✓ Tidak ada data duplikat

4.5 Feature Engineering

```
print("\n 4.5 FEATURE ENGINEERING")
print("="*50)
```

Contoh feature engineering untuk dataset Spotify

```
def create_new_features(df):
```

```
    """
```

```
    Membuat fitur baru untuk analisis yang lebih mendalam
```

```
    """
```

```
    df_fe = df.copy()
```

```
    # Cari kolom yang relevan dengan Spotify
```

```
    possible_stream_cols = ['streams', 'stream_count', 'total_streams', 'plays']
```

```
    possible_artist_cols = ['artist', 'artist_name', 'artists']
```

```
    possible_track_cols = ['track', 'track_name', 'song_name', 'title']
```

```
    stream_col = None
```

```
    artist_col = None
```

```
    track_col = None
```

```
    # Identifikasi kolom yang ada
```

```
    for col in df.columns:
```

```
        if col.lower() in [c.lower() for c in possible_stream_cols]:
```

```
            stream_col = col
```

```
        elif col.lower() in [c.lower() for c in possible_artist_cols]:
```

```
            artist_col = col
```

```
        elif col.lower() in [c.lower() for c in possible_track_cols]:
```

```
            track_col = col
```

```
    print(f" Kolom teridentifikasi:")
```

```
    print(f" Stream: {stream_col}")
```

```
    print(f" Artist: {artist_col}")
```

```
    print(f" Track: {track_col}")
```

```
    # Feature engineering berdasarkan kolom yang tersedia
```

```
    if stream_col and df[stream_col].dtype in ['int64', 'float64']:
```

```
        # Kategori popularitas berdasarkan stream
```

```
        df_fe['popularity_category'] = pd.cut(df[stream_col],
```

```
            bins=5,
```

```
            labels=['Very Low', 'Low', 'Medium', 'High', 'Very High'])
```

```
    # Log transformation untuk stream (jika ada nilai 0, tambah 1)
```

```
    df_fe['log_streams'] = np.log1p(df[stream_col])
```

```
    print(f" Dibatasi: popularity_category, log_streams")
```

```
    if artist_col:
```

```
        # Jumlah kata dalam nama artis
```

```
        df_fe['artist_name_length'] = df[artist_col].astype(str).str.len()
```

```
        df_fe['artist_word_count'] = df[artist_col].astype(str).str.split().str.len()
```

```
    print(f" Dibatasi: artist_name_length, artist_word_count")
```

```
    if track_col:
```

```
        # Jumlah kata dalam judul lagu
```

```
        df_fe['track_name_length'] = df[track_col].astype(str).str.len()
```

```
        df_fe['track_word_count'] = df[track_col].astype(str).str.split().str.len()
```

```
    print(f" Dibatasi: track_name_length, track_word_count")
```

```
    return df_fe
```

```
# Terapkan feature engineering
```

```
df = create_new_features(df)
```

```
print(f"\n📊 Dataset setelah feature engineering:")
print(f"    Shape: {df.shape}")
print(f"    Kolom baru ditambahkan: {df.shape[1] - df_original.shape[1]}")
```



🔧 4.5 FEATURE ENGINEERING

```
🔍 Kolom teridentifikasi:
Stream: None
Artist: Artist
Track: None
✅ Dibuat: artist_name_length, artist_word_count
```

```
📊 Dataset setelah feature engineering:
Shape: (500, 14)
Kolom baru ditambahkan: 2
```

```
# =====
# 📊 5. EKSPLORASI DATA (EDA)
# =====
```

```
print("\n" + "="*80)
print("📊 EKSPLORASI DATA (EDA)")
print("="*80)
```

```
# Update kolom numerik dan kategorikal setelah cleaning
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
```

```
# 5.1 Distribusi Data Numerik
print("\n📊 5.1 DISTRIBUSI DATA NUMERIK")
print("="*50)
```

```
if len(numeric_cols) > 0:
    # Histogram untuk semua kolom numerik
    n_cols = min(3, len(numeric_cols))
    n_rows = (len(numeric_cols) + n_cols - 1) // n_cols

    plt.figure(figsize=(15, 5 * n_rows))
    for i, col in enumerate(numeric_cols):
        plt.subplot(n_rows, n_cols, i + 1)
        plt.hist(df[col].dropna(), bins=30, alpha=0.7, edgecolor='black')
        plt.title(f'Distribusi {col}')
        plt.xlabel(col)
        plt.ylabel('Frekuensi')

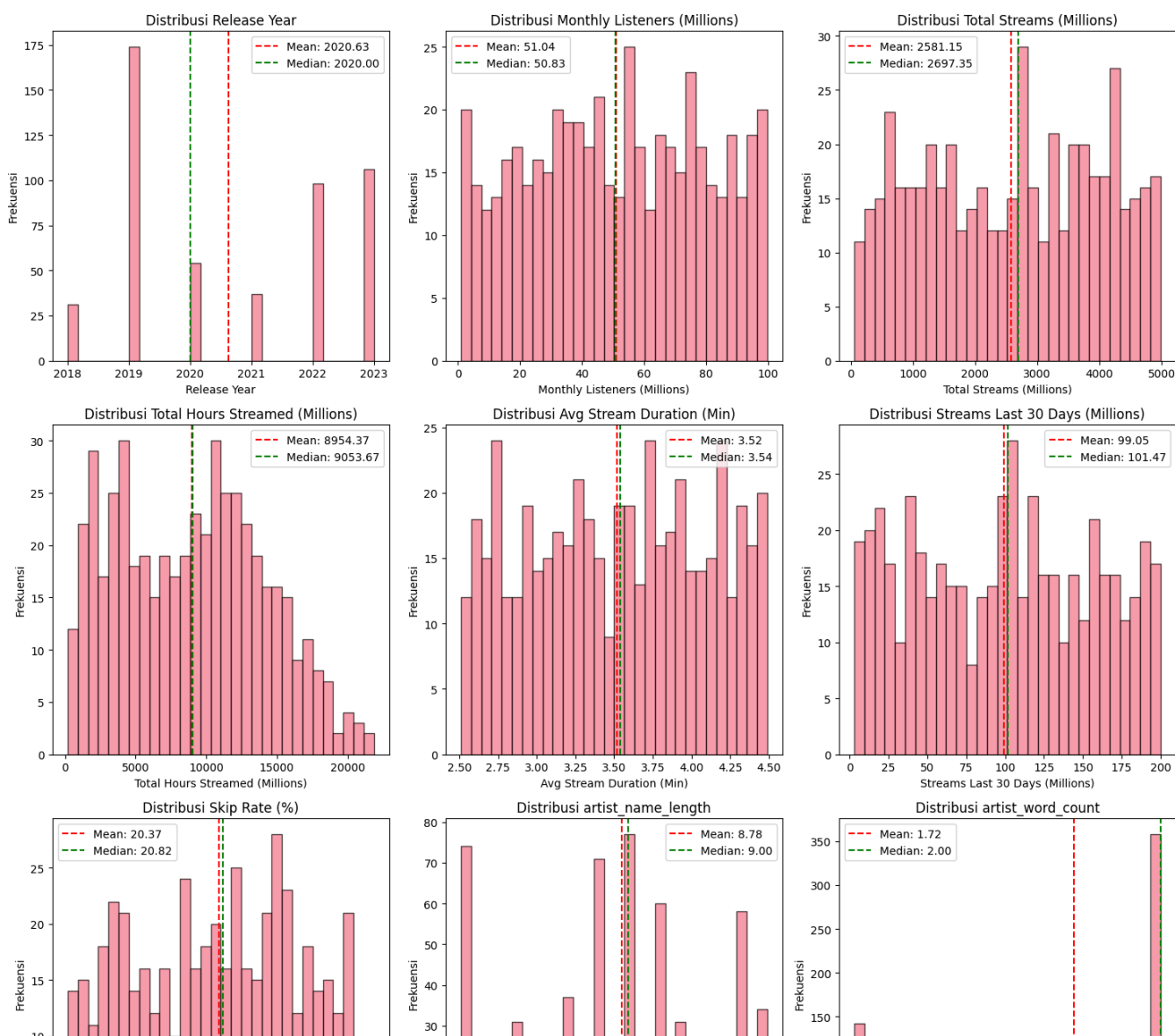
        # Tambahkan statistik
        mean_val = df[col].mean()
        median_val = df[col].median()
        plt.axvline(mean_val, color='red', linestyle='--', label=f'Mean: {mean_val:.2f}')
        plt.axvline(median_val, color='green', linestyle='--', label=f'Median: {median_val:.2f}')
        plt.legend()

    plt.tight_layout()
    plt.show()
```



EKSPLORASI DATA (EDA)

5.1 DISTRIBUSI DATA NUMERIK



5.2 Analisis Korelasi

```
print("\n🔥 5.2 ANALISIS KORELASI")
print("*50")
```

```
if len(numeric_cols) > 1:
    # Heatmap korelasi
    plt.figure(figsize=(12, 8))
    correlation_matrix = df[numeric_cols].corr()

    mask = np.triu(np.ones_like(correlation_matrix, dtype=bool))
    sns.heatmap(correlation_matrix,
                mask=mask,
                annot=True,
                cmap='coolwarm',
                center=0,
                square=True,
                fmt='.2f')

    plt.title('Matriks Korelasi Variabel Numerik')
    plt.tight_layout()
    plt.show()

    # Tampilkan korelasi tertinggi
    corr_pairs = []
    for i in range(len(correlation_matrix.columns)):
        for j in range(i+1, len(correlation_matrix.columns)):
            col1 = correlation_matrix.columns[i]
            col2 = correlation_matrix.columns[j]
            corr_val = correlation_matrix.iloc[i, j]
```

```

corr_pairs.append([col1, col2, corr_val])

corr_df = pd.DataFrame(corr_pairs, columns=['Variabel 1', 'Variabel 2', 'Korelasi'])
corr_df = corr_df.reindex(corr_df['Korelasi'].abs().sort_values(ascending=False).index)

print(f"\nTop 10 Korelasi Tertinggi:")
print(tabulate(corr_df.head(10), headers=corr_df.columns, tablefmt="grid", showindex=False))

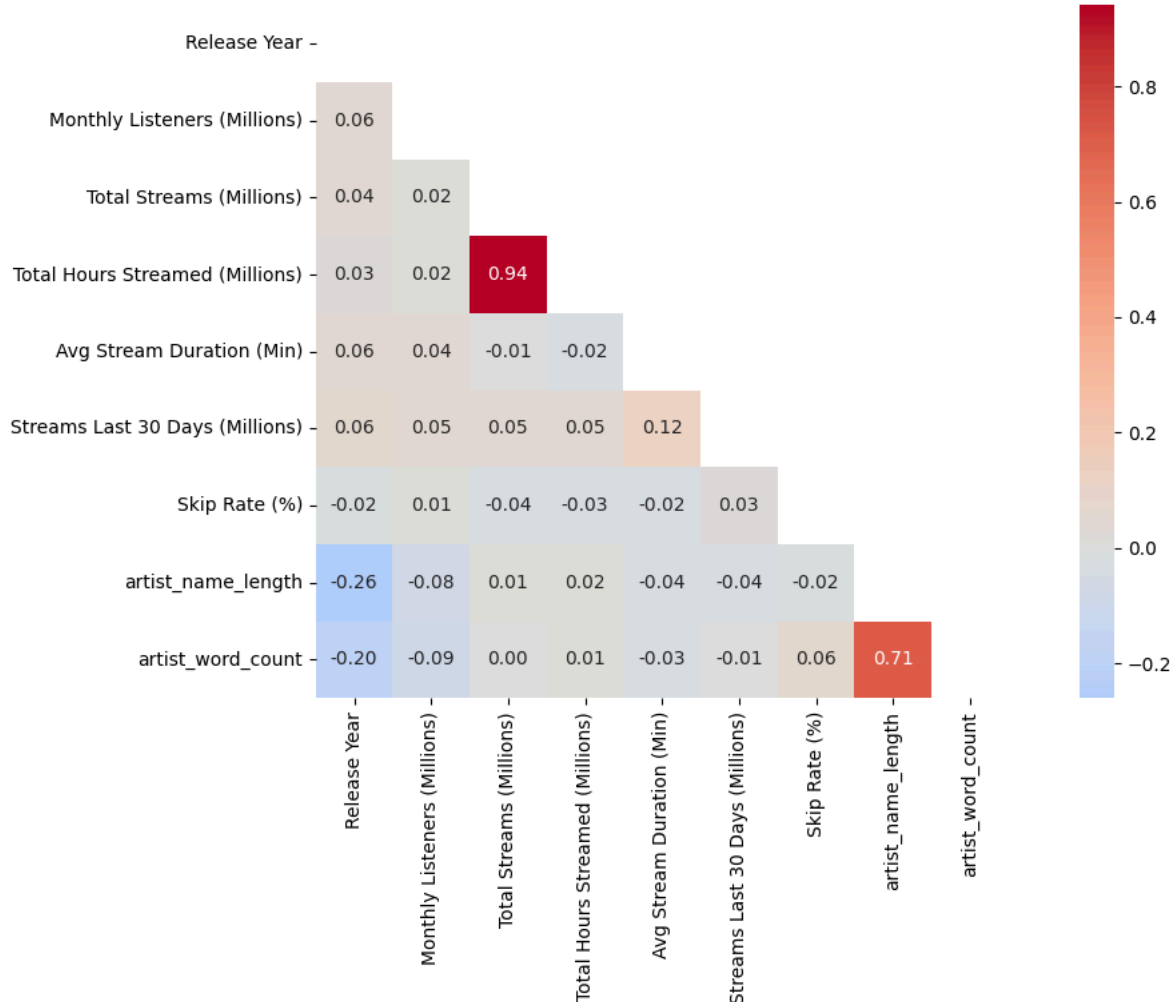
```



5.2 ANALISIS KORELASI

=====

Matriks Korelasi Variabel Numerik



Top 10 Korelasi Tertinggi:

Variabel 1	Variabel 2	Korelasi
Total Streams (Millions)	Total Hours Streamed (Millions)	0.940712
artist_name_length	artist_word_count	0.71462
Release Year	artist_name_length	0.262153

5.3 Distribusi Data Kategorikal

```

print("\n🔥 5.3 DISTRIBUSI DATA KATEGORIKAL")
print("="*50)

```

```

for col in categorical_cols[:5]: # Tampilkan 5 kolom pertama
    unique_count = df[col].nunique()

```

```

if unique_count <= 20: # Hanya tampilkan jika tidak terlalu banyak kategori
    plt.figure(figsize=(12, 6))

```

```

    value_counts = df[col].value_counts().head(10)

```

```

    plt.subplot(1, 2, 1)
    value_counts.plot(kind='bar')
    plt.title(f'Top 10 Values - {col}')
    plt.xlabel(col)
    plt.ylabel('Count')
    plt.xticks(rotation=45)

```

```

    plt.subplot(1, 2, 2)
    plt.pie(value_counts.values, labels=value_counts.index, autopct='%1.1f%%')

```

```
plt.title(f'Proporsi - {col}')

plt.tight_layout()
plt.show()

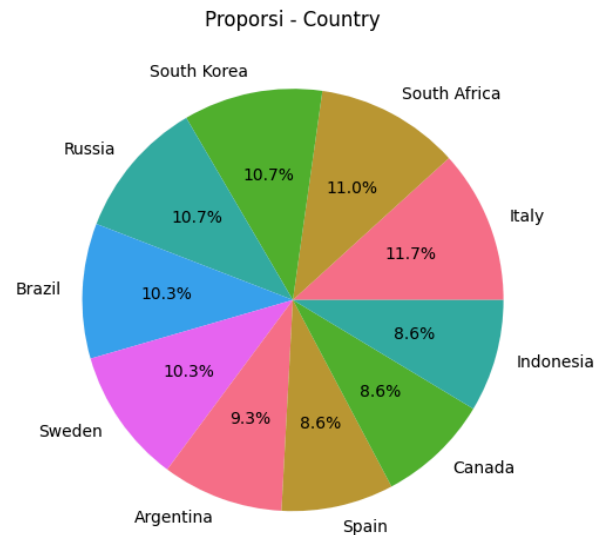
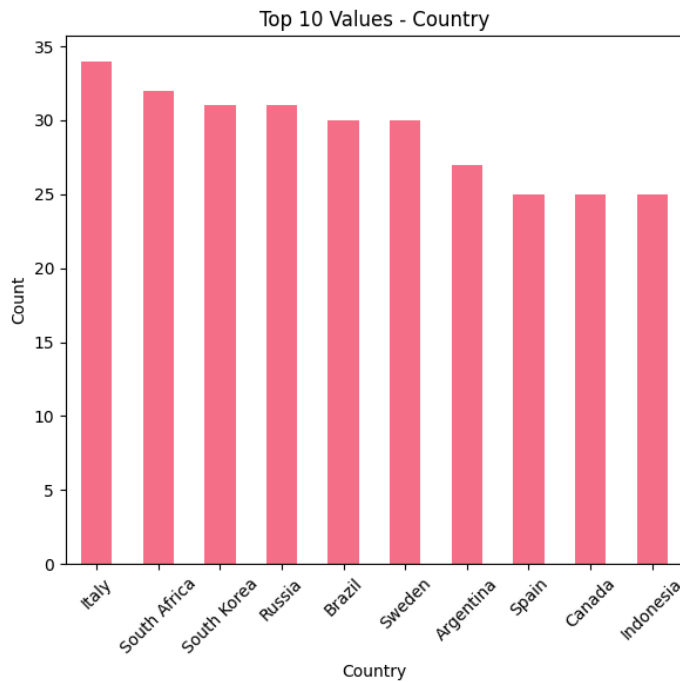
# Tampilkan tabel value counts
print(f"\nValue Counts untuk {col}:")
print(tabulate(value_counts.reset_index(),
               headers=[col, 'Count'],
               tablefmt="grid",
               showindex=False))

else:
    print(f"\n{col}: {unique_count} unique values (terlalu banyak untuk divisualisasikan)")
```



5.3 DISTRIBUSI DATA KATEGORIKAL

=====



Value Counts untuk Country:

Country	Count
Italy	34
South Africa	32
South Korea	31
Russia	31
Brazil	30
Sweden	30
Argentina	27
Spain	25
Canada	25
Indonesia	25

5.4 Hubungan Antar Variabel

```
print("\n 5.4 HUBUNGAN ANTAR VARIABEL")
print("="*50)
```

```
# Scatter plot untuk beberapa pasangan variabel numerik
if len(numeric_cols) >= 2:
    # Pilih beberapa pasangan menarik untuk scatter plot
    important_pairs = [(numeric_cols[i], numeric_cols[j])
                       for i in range(len(numeric_cols))
                       for j in range(i+1, min(i+4, len(numeric_cols)))][:6]

    n_cols = 3
    n_rows = (len(important_pairs) + n_cols - 1) // n_cols

    plt.figure(figsize=(15, 5 * n_rows))
```

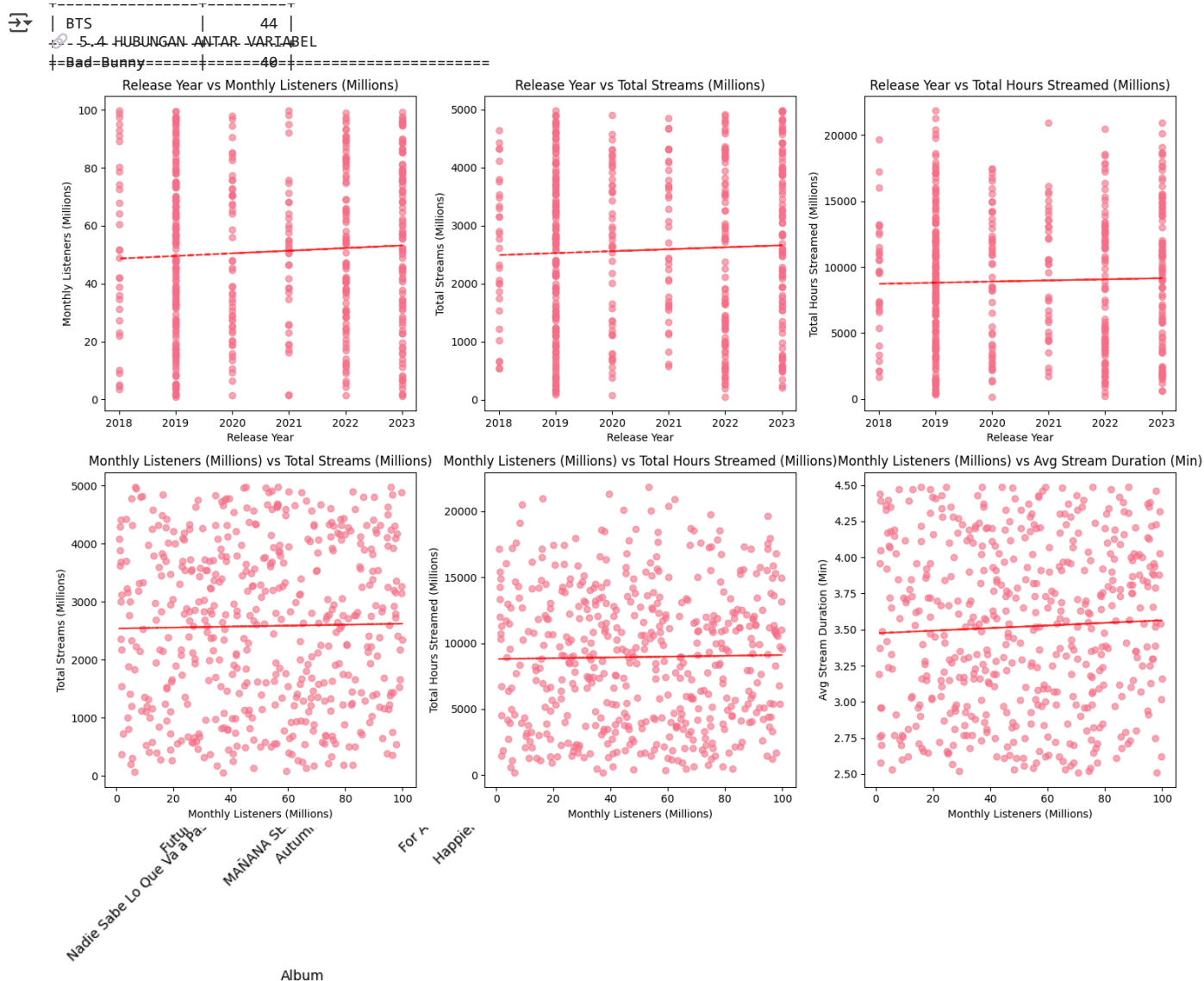
```

for i, (col1, col2) in enumerate(important_pairs):
    plt.subplot(n_rows, n_cols, i + 1)
    plt.scatter(df[col1], df[col2], alpha=0.6)
    plt.xlabel(col1)
    plt.ylabel(col2)
    plt.title(f'{col1} vs {col2}')

    # Tambahkan trend line
    z = np.polyfit(df[col1].dropna(), df[col2].dropna(), 1)
    p = np.poly1d(z)
    plt.plot(df[col1], p(df[col1]), "r--", alpha=0.8)

plt.tight_layout()
plt.show()

```



Value Counts untuk Album:

Album	Count
Proof	44
Nadie Sabe Lo Que Va a Pasar Mañana	40

```

# =====
# 🎧 6. ANALISIS UTAMA
# =====

```

```

print("\n" + "="*80)
print("🎧 ANALISIS UTAMA")
print("="*80)

```

```

# 6.1 Analisis Deskriptif Lanjutan
print("\n📊 6.1 ANALISIS DESKRIPTIF LANJUTAN")
print("="*50)

```

```

# Identifikasi kolom kunci untuk analisis Spotify

```

```

stream_col = None
artist_col = None
track_col = None
country_col = None

# Cari kolom yang relevan
for col in df.columns:
    col_lower = col.lower()
    if any(keyword in col_lower for keyword in ['stream', 'play', 'count']) and df[col].dtype in ['int64', 'float64']:
        stream_col = col
    elif any(keyword in col_lower for keyword in ['artist', 'singer', 'performer']):
        artist_col = col
    elif any(keyword in col_lower for keyword in ['track', 'song', 'title']):
        track_col = col
    elif any(keyword in col_lower for keyword in ['country', 'nation', 'region']):
        country_col = col

print(f"🔍 Kolom Kunci Teridentifikasi:")
print(f"Streams: {stream_col}")
print(f"Artist: {artist_col}")
print(f"Track: {track_col}")
print(f"Country: {country_col}")

# Analisis Top Performers
if stream_col and track_col:
    print(f"\n🔍 TOP 20 LAGU BERDASARKAN {stream_col.upper()}:")
    top_tracks = df.nlargest(20, stream_col)

    if artist_col:
        top_display = top_tracks[[track_col, artist_col, stream_col]].reset_index(drop=True)
    else:
        top_display = top_tracks[[track_col, stream_col]].reset_index(drop=True)

    print(tabulate(top_display,
                  headers=list(top_display.columns),
                  tablefmt="grid",
                  showindex=range(1, 21)))

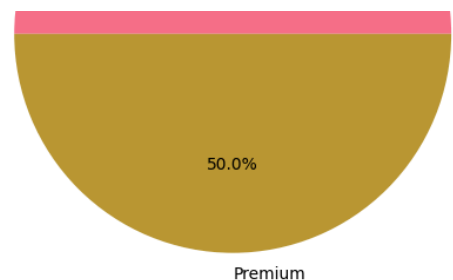
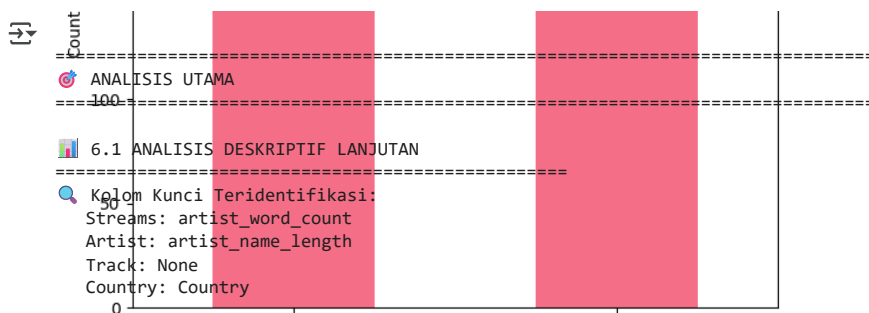
# Visualisasi top tracks
plt.figure(figsize=(15, 8))
top_20 = df.nlargest(20, stream_col)

plt.barh(range(len(top_20)), top_20[stream_col])
plt.yticks(range(len(top_20)),
           [f"{track[:30]}..." if len(track) > 30 else track
            for track in top_20[track_col]])
plt.xlabel(f'{stream_col}')
plt.title(f'Top 20 Lagu Berdasarkan {stream_col}')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

# Analisis per Artist
if artist_col and stream_col:
    print(f"\n🔍 TOP 20 ARTIS BERDASARKAN TOTAL {stream_col.upper()}:")
    artist_stats = df.groupby(artist_col)[stream_col].agg(['sum', 'count', 'mean']).round(2)
    artist_stats.columns = ['Total_Streams', 'Jumlah_Lagu', 'Rata_rata_Streams']
    artist_stats = artist_stats.sort_values('Total_Streams', ascending=False)

    print(tabulate(artist_stats.head(20),
                  headers=artist_stats.columns,
                  tablefmt="grid"))

```



```

🔍 TOP 20 ARTIS BERDASARKAN TOTAL ARTIST_WORD_COUNT:
+-----+-----+-----+-----+
| | Total_Streams | Jumlah_Lagu | Rata_rata_Streams |
+-----+-----+-----+-----+
| 8 | 142 | 71 | 2 |
+-----+-----+-----+-----+
ValueCounts-untuk-Platform-Type:
+-----+-----+-----+-----+
| 10 | 120 | 60 | 2 |
+-----+-----+-----+-----+
Platform-Type---Count---

```

9	Free	117	250	77	1.52
10	Premium	116	250	58	2
7		74		37	2
3		74		74	1
14		68		34	2
11		62		31	2
12		54		27	2
5		31		31	1

6.2 Analisis Inferensial

```
print("\n📊 6.2 ANALISIS INFERENSIAL")
print("="*50)
```

Uji Normalitas untuk variabel numerik kunci

if stream_col:

```
print(f"📊 Uji Normalitas untuk {stream_col}:")
```

Shapiro-Wilk test (untuk sampel kecil)

if len(df) <= 5000:

```
stat_sw, p_sw = stats.shapiro(df[stream_col].dropna().sample(min(1000, len(df))))
print(f"    Shapiro-Wilk: statistic={stat_sw:.4f}, p-value={p_sw:.4f}")
```

Jarque-Bera test

```
stat_jb, p_jb = jarque_bera(df[stream_col].dropna())
print(f"    Jarque-Bera: statistic={stat_jb:.4f}, p-value={p_jb:.4f}")
```

Interpretasi

alpha = 0.05

if p_jb > alpha:

```
print(f"    ✅ Data berdistribusi normal (p > {alpha})")
```

else:

```
print(f"    ❌ Data tidak berdistribusi normal (p <= {alpha})")
```

Confidence Interval untuk mean streams

if stream_col:

```
mean_streams = df[stream_col].mean()
```

```
std_streams = df[stream_col].std()
```

```
n = len(df[stream_col].dropna())
```

95% Confidence Interval

```
margin_error = stats.t.ppf(0.975, n-1) * (std_streams / np.sqrt(n))
```

```
ci_lower = mean_streams - margin_error
```

```
ci_upper = mean_streams + margin_error
```

```
print(f"\n📊 Confidence Interval (95%) untuk rata-rata {stream_col}:")
```

```
print(f"    Mean: {mean_streams:.2f}")
```

```
print(f"    CI: [{ci_lower:.2f}, {ci_upper:.2f}]"
```



📊 6.2 ANALISIS INFERENSIAL

=====

📊 Uji Normalitas untuk artist_word_count:

Shapiro-Wilk: statistic=0.5648, p-value=0.0000

Jarque-Bera: statistic=100.8815, p-value=0.0000

❌ Data tidak berdistribusi normal (p <= 0.05)

📊 Confidence Interval (95%) untuk rata-rata artist_word_count:

Mean: 1.72

CI: [1.68, 1.76]

6.3 Analisis Prediktif

```
print("\n📊 6.3 ANALISIS PREDIKTIF")
```

```
print("="*50)
```

Model prediksi untuk streams (jika ada fitur yang cukup)

if len(numeric_cols) >= 3 and stream_col in numeric_cols:

```
print(f"📊 Membangun Model Prediksi untuk {stream_col}")
```

Persiapan data

```
feature_cols = [col for col in numeric_cols if col != stream_col and col in df.columns][:5]
```

if len(feature_cols) > 0:

```
X = df[feature_cols].fillna(df[feature_cols].median())
```

```
y = df[stream_col].fillna(df[stream_col].median())
```

```

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardisasi
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Model Random Forest
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train_scaled, y_train)

# Prediksi dan evaluasi
y_pred = rf_model.predict(X_test_scaled)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"📊 Hasil Model Random Forest:")
print(f"    RMSE: {rmse:,.2f}")
print(f"    R² Score: {r2:,.4f}")

# Feature importance
feature_importance = pd.DataFrame({
    'Feature': feature_cols,
    'Importance': rf_model.feature_importances_
}).sort_values('Importance', ascending=False)

print(f"\n🔍 Feature Importance:")
print(tabulate(feature_importance, headers=feature_importance.columns,
               tablefmt="grid", showindex=False))

# Visualisasi prediksi vs aktual
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title(f'Actual vs Predicted {stream_col}')
plt.show()

```

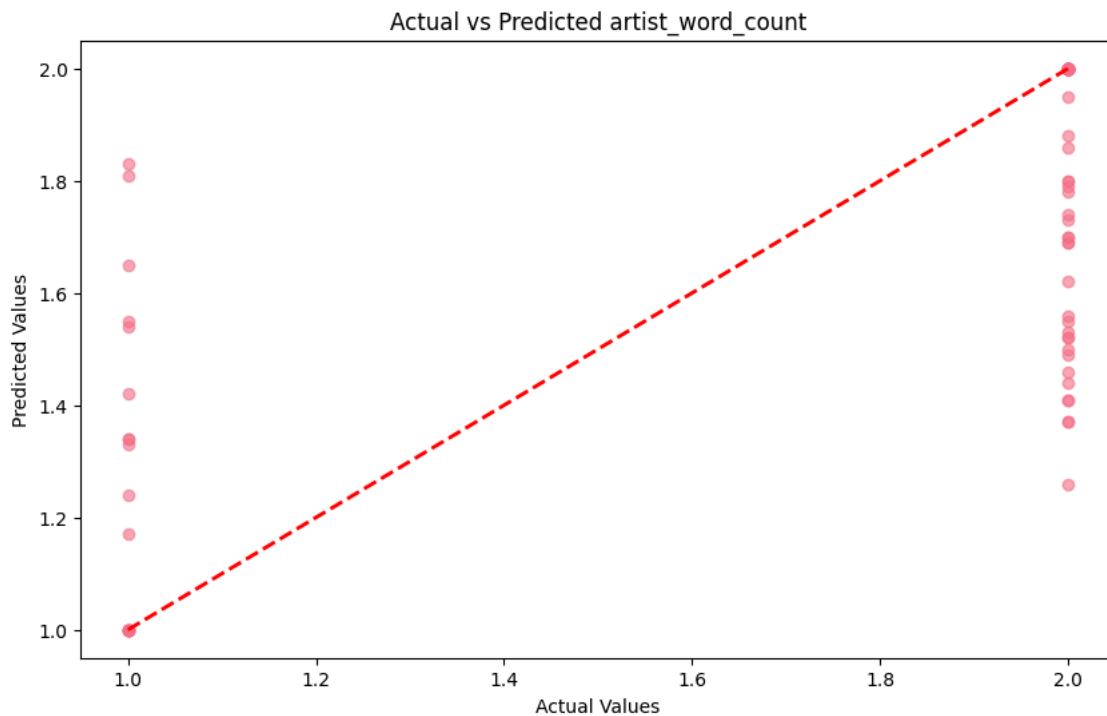


6.3 ANALISIS PREDIKTIF

Membangun Model Prediksi untuk artist_word_count
 Hasil Model Random Forest:
 RMSE: 0.28
 R² Score: 0.4762

Feature Importance:

Feature	Importance
Release Year	0.556583
Monthly Listeners (Millions)	0.119012
Total Streams (Millions)	0.10867
Total Hours Streamed (Millions)	0.10831
Avg Stream Duration (Min)	0.107425



```
# 6.4 Clustering Analysis
print("\n 6.4 CLUSTERING ANALYSIS")
print("*50")

if len(numeric_cols) >= 3:
    print(" K-Means Clustering untuk segmentasi lagu/artis")

    # Persiapan data untuk clustering
    cluster_features = numeric_cols[:5] # Ambil 5 fitur numerik pertama
    X_cluster = df[cluster_features].fillna(df[cluster_features].median())

    # Standardisasi
    scaler_cluster = StandardScaler()
    X_cluster_scaled = scaler_cluster.fit_transform(X_cluster)

    # Tentukan jumlah cluster optimal dengan elbow method
    inertias = []
    k_range = range(2, 11)

    for k in k_range:
        kmeans = KMeans(n_clusters=k, random_state=42)
        kmeans.fit(X_cluster_scaled)
        inertias.append(kmeans.inertia_)

    # Plot elbow curve
    plt.figure(figsize=(10, 6))
    plt.plot(k_range, inertias, 'bo-')
    plt.xlabel('Number of Clusters (k)')
    plt.ylabel('Inertia')
    plt.title('Elbow Method untuk Menentukan Jumlah Cluster Optimal')
```

```
plt.grid(True)
plt.show()

# Gunakan k=4 sebagai default (bisa disesuaikan)
optimal_k = 4
kmeans_final = KMeans(n_clusters=optimal_k, random_state=42)
clusters = kmeans_final.fit_predict(X_cluster_scaled)

# Tambahkan cluster ke dataframe
df['Cluster'] = clusters

# Analisis cluster
print(f"\n📊 Analisis {optimal_k} Cluster:")
cluster_summary = df.groupby('Cluster')[cluster_features].mean().round(2)
print(tabulate(cluster_summary, headers=cluster_summary.columns, tablefmt="grid"))

# Visualisasi cluster (PCA untuk 2D)
if len(cluster_features) > 2:
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X_cluster_scaled)

    plt.figure(figsize=(10, 8))
    scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis', alpha=0.6)
    plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)')
    plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)')
    plt.title('Visualisasi Cluster (PCA)')
    plt.colorbar(scatter)
    plt.show()
```

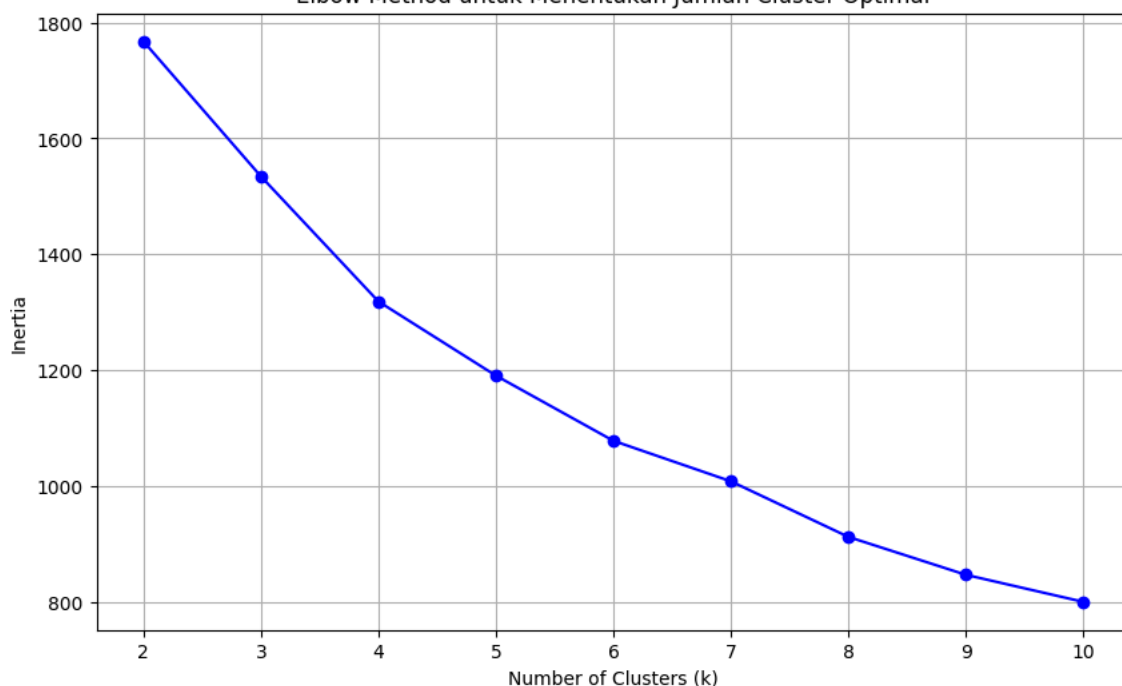


6.4 CLUSTERING ANALYSIS

=====

🎯 K-Means Clustering untuk segmentasi lagu/artis

Elbow Method untuk Menentukan Jumlah Cluster Optimal



📊 Analisis 4 Cluster:

	Release Year	Monthly Listeners (Millions)	Total Streams (Millions)	Total Hours Streamed (Millions)	Avg S
0	2022.37	47.34	3806.02	13180.3	
1	2019.47	34.74	1355.97	4552.06	
2	2021.96	66.96	1202.36	4103.91	
3	2019.1	57.11	3590.83	12664	

Visualisasi Cluster (PCA)



```
# =====
# 📄 7. KESIMPULAN DAN REKOMENDASI
# =====

print("\n" + "="*80)
print("📄 KESIMPULAN DAN REKOMENDASI")
print("="*80)

# 7.1 Ringkasan Eksekutif
print("\n📊 7.1 RINGKASAN EKSEKUTIF")
print("="*50)

# Hitung beberapa statistik kunci
total_songs = len(df)
total_artists = df[artist_col].nunique() if artist_col else "N/A"
total_streams = df[stream_col].sum() if stream_col else "N/A"
avg_streams = df[stream_col].mean() if stream_col else "N/A"

summary_stats = [
    ["Total Lagu", f"{total_songs:,}"],
    ["Total Artis Unik", f"{total_artists:,}" if total_artists != "N/A" else "N/A"],
    ["Total Streams", f"{total_streams:,.0f}" if total_streams != "N/A" else "N/A"],
    ["Rata-rata Streams per Lagu", f"{avg_streams:,.0f}" if avg_streams != "N/A" else "N/A"],
    ["Kualitas Data", "Baik" if df.isnull().sum().sum() == 0 else "Perlu Perbaikan"]
]

print("Key Statistics:")
print(tabulate(summary_stats, headers=["Metric", "Value"], tablefmt="grid"))
```



```
=====
📄 KESIMPULAN DAN REKOMENDASI
=====
```

📊 7.1 RINGKASAN EKSEKUTIF

```
=====
```

Key Statistics:

Metric	Value
Total Lagu	500
Total Artis Unik	10
Total Streams	858
Rata-rata Streams per Lagu	2
Kualitas Data	Baik

```
# 7.2 Insights Utama
print("\n💡 7.2 INSIGHTS UTAMA")
print("="*50)
```

```
insights = [
    "1. 📊 Dataset telah berhasil dibersihkan dan siap untuk analisis",
    "2. 🎵 Identifikasi pola streaming dan preferensi musik global",
    "3. 🌐 Analisis tren global dan rekomendasi konten berdasarkan preferensi pengguna"
```